

# دليل أوامر التشغيل والطوارئ والنسخ الاحتياطي

Runbook عملي لإدارة Goalix في التطوير والإنتاج: تشغيل الخدمات، إصلاح الحالات الشائعة، أول أدمن بعد داتا بيز فاضية، backup/restore، الاختبارات، والجودة.

النطاق

**Dev + Production +  
DR**

آخر تحديث

**July 2026 07**

المشروع

**Goalix**

## قواعد تشغيل سريعة قبل أي أمر

اعتبر هذا الدليل نقطة البداية وقت الضغط. اقرأ الحالة، نفذ الأمر، ثم راقب اللوجات. أوامر restore وbootstrap مؤثرة ويجب تنفيذها من شخص موثوق.

### MFA للأدمن والمدرّب

أول أدمن يتم إنشاؤه بالـ bootstrap ثم يجبره النظام على إعداد MFA قبل استخدام admin panel بالكامل.

### Worker هو مكان الأتمتة

الـ backup التلقائي، cleanup، lifecycle، queues يجب أن تعمل على worker وليس API container.

### لا تعمل Restore بلا Backup

زر restore والـ CLI destructive. النظام ينشئ safety backup قبل restore من الواجهة، لكن في الطوارئ خذ نسخة يدوية لو أمكن.

**مسار التشغيل الطبيعي:** env جاهز -> bootstrap admin -> migrations عند الحاجة -> worker شغال -> verify -> backup automatic قبل deploy.

**تحذير:** لو الداتابيز اتمسحت بالكامل، زر Settings لن يفتح لأن login نفسه يعتمد على الداتابيز. استخدم restore CLI من السيرفر أو Docker worker.

## 1.1 تشغيل التطوير المحلي

أوامر وحالات جاهزة للتنفيذ.

## 2.2 تشغيل الإنتاج Docker

أوامر وحالات جاهزة للتنفيذ.

## 3.3 Backend Database

أوامر وحالات جاهزة للتنفيذ.

## 4.4 أول Admin بعد داتابيز فاضية

أوامر وحالات جاهزة للتنفيذ.

## 5.5 Restore Backup

أوامر وحالات جاهزة للتنفيذ.

## 6.6 Data Lifecycle Jobs

أوامر وحالات جاهزة للتنفيذ.

## 7.7 الجودة والاختبارات والأمان

أوامر وحالات جاهزة للتنفيذ.

## 8.8 Deploy وأدوات مساعدة

أوامر وحالات جاهزة للتنفيذ.

# 1. تشغيل التطوير المحلي

| الحالة / الهدف        | الأمر                               | ملاحظات التشغيل                                                    |
|-----------------------|-------------------------------------|--------------------------------------------------------------------|
| تجهيز Redis فقط       | <code>npm run infra:up</code>       | يشغل Redis من docker- على localhost:6379 compose.dev.yml           |
| إيقاف Redis المحلي    | <code>npm run infra:down</code>     | يوقف خدمات التطوير ويحافظ على أو يسمح حسب Docker compose behavior. |
| حالة Redis            | <code>npm run infra:status</code>   | يعرض حالة container الخاص بـ Redis.                                |
| تحرير بورتات التطوير  | <code>npm run free:dev-ports</code> | مفيد لو 3000/3001/6379 متعلقة من تشغيل سابق.                       |
| تجهيز موديلات Python  | <code>npm run models:setup</code>   | يتأكد من جاهزية ملفات ومتطلبات نماذج الذكاء الاصطناعي.             |
| تشغيل النظام كامل Dev | <code>npm run dev</code>            | يشغل + frontend + backend المتطلبات حسب scripts/dev.js             |
| تشغيل HTTPS Dev       | <code>npm run dev:https</code>      | نفس dev لكن بشهادات HTTPS للتجارب التي تحتاج Secure Context.       |
| تشغيل Frontend فقط    | <code>npm run dev:frontend</code>   | Next.js فقط على 0.0.0.0:3001                                       |
| تشغيل Backend فقط     | <code>npm run dev:backend</code>    | يدخل golx-backend ويشغل nodemon.                                   |
| عرض روابط LAN         | <code>npm run lan:info</code>       | يعرض روابط الشبكة المحلية بعد التشغيل.                             |
| تجهيز LAN access      | <code>npm run lan:setup</code>      | يضبط إعدادات ويندوز/Firewall حسب السكربت.                          |

## 2. تشغيل الإنتاج Docker

| الحالة / الهدف        | الأمر                                                                                    | ملاحظات التشغيل                                                                    |
|-----------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| بناء وتشغيل الإنتاج   | <code>docker compose -f docker-compose.prod.yml up -d --build</code>                     | يبنى frontend/backend ويشغل postgres, redis, migrate, api, worker, frontend, nginx |
| عرض حالة الخدمات      | <code>docker compose -f docker-compose.prod.yml ps</code>                                | أول أمر تشخيص بعد deploy.                                                          |
| متابعة اللوجات        | <code>docker compose -f docker-compose.prod.yml logs -f api worker frontend nginx</code> | راقب الإقلاع، ال health checks، وأخطاء migrations.                                 |
| إعادة تشغيل API       | <code>docker compose -f docker-compose.prod.yml restart api</code>                       | لو غيرت env أو الخدمة عقلت.                                                        |
| إعادة تشغيل worker    | <code>docker compose -f docker-compose.prod.yml restart worker</code>                    | لو queues أو automations وقفت.                                                     |
| إيقاف الإنتاج         | <code>docker compose -f docker-compose.prod.yml down</code>                              | يوقف containers بدون حذف volumes.                                                  |
| تشغيل migration يدوي  | <code>docker compose -f docker-compose.prod.yml run --rm migrate</code>                  | يشغل خدمة migrate مرة واحدة.                                                       |
| فتح shell داخل API    | <code>docker compose -f docker-compose.prod.yml exec api sh</code>                       | للتشخيص من داخل الشبكة الداخلية.                                                   |
| فتح shell داخل worker | <code>docker compose -f docker-compose.prod.yml exec worker sh</code>                    | مفيد لل backup و jobs.                                                             |

## Database و Backend .3

| الحالة / الهدف                | الأمر                                                     | ملاحظات التشغيل                                |
|-------------------------------|-----------------------------------------------------------|------------------------------------------------|
| تشغيل backend dev             | cd golx-backend && npm run dev                            | nodemon على src/server.js                      |
| تشغيل backend محلي production | cd golx-backend && npm start                              | يشغل src/server.js مباشرة.                     |
| تشغيل worker                  | cd golx-backend && npm run worker                         | يشغل BullMQ workers وال background automations |
| تطبيق migrations              | cd golx-backend && npm run migrate                        | لازم بعد أي migration جديد.                    |
| Rollback آخر migration        | cd golx-backend && npm run migrate:rollback               | استخدمه بحذر في staging أكثر من production.    |
| إنشاء migration               | cd golx-backend && npm run migrate:make -- migration_name | لإضافة migration جديدة.                        |
| تشغيل كل seeds                | cd golx-backend && npm run seed                           | يشغل seeds من فولدر seeds.                     |
| Seed أول Admin فقط            | cd golx-backend && npm run seed:admin                     | بدل legacy يعتمد على bootstrap admin الجديد.   |
| اختبار backend                | cd golx-backend && npm test                               | Jest runInBand                                 |
| Lint backend                  | cd golx-backend && npm run lint                           | ESLint على src                                 |
| Audit بنية الداتابيز          | cd golx-backend && npm run db:audit                       | يفحص مؤشرات/معمارية الداتابيز.                 |
| Audit صارم                    | cd golx-backend && npm run db:audit:strict                | يفشل عند مشاكل معمارية معينة.                  |

## 4. أول Admin بعد داتابيز فاضية

| الحالة / الهدف                      | الأمر                                                                                                                                                                                                                                                          | ملاحظات التشغيل                                                                     |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| إنشاء أول Academy Owner             | <pre>\$env:BOOTSTRAP_ADMIN_EMAIL="owner@example.com";<br/>\$env:BOOTSTRAP_ADMIN_USERNAME="admin";<br/>;<br/>\$env:BOOTSTRAP_ADMIN_PASSWORD="StrongPass#123";<br/>\$env:BOOTSTRAP_ACADEMY_NAME="Goalix Academy"; cd golx-backend; npm run bootstrap:admin</pre> | ينشئ الأكاديمية، أول أدمن، IAM roles، admin_accounts، ويترك MFA مطلوب على أول دخول. |
| Recovery لو أدمن موجود وتريد تحديثه | <pre>\$env:BOOTSTRAP_ADMIN_ALLOW_EXISTING="true"; npm run bootstrap:admin</pre>                                                                                                                                                                                | استخدمه فقط لو أنت متأكد أنه recovery شرعي.                                         |
| Docker bootstrap                    | <pre>docker compose -f docker-compose.prod.yml run --rm -e<br/>BOOTSTRAP_ADMIN_PASSWORD="StrongPass#123" -e<br/>BOOTSTRAP_ADMIN_EMAIL="owner@example.com" -e<br/>BOOTSTRAP_ACADEMY_NAME="Goalix Academy" api npm run bootstrap:admin</pre>                     | ينفذ bootstrap داخل شبكة Docker بعد migrations.                                     |
| بعد إنشاء الأدمن                    | افتح admin-login/ ثم فعل MFA من admin/settings/                                                                                                                                                                                                                | النظام يسمح بالدخول الأول لكنه يقفل admin panel على Settings حتى تفعيل 2FA.         |

## Restore و Backup .5

| الحالة / الهدف                      | الأمر                                                                                                                                                      | ملاحظات التشغيل                                                      |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Backup يدوي محلي</b>             | <pre>cd golx-backend; \$env:DATABASE_URL="postgresql://USER: PASSWORD@HOST:5432/golx_main"; \$env:BACKUP_DIR="D:\\goalix-backups"; npm run backup:db</pre> | ينتج dump و sha256.                                                  |
| <b>Backup داخل Docker</b>           | <pre>docker compose -f docker- compose.prod.yml exec worker npm run backup:db</pre>                                                                        | يكتب داخل app/backups/ على volume goalix-backups.                    |
| <b>Restore من CLI</b>               | <pre>cd golx-backend; \$env:CONFIRM_RESTORE="RESTORE GOALIX"; npm run restore:db -- golx_golx_main_auto_2026-07-07T10-00- 00.dump</pre>                    | يستخدم pg_restore --clean --if-exists --destructive لعملية.          |
| <b>Restore داخل Docker</b>          | <pre>docker compose -f docker- compose.prod.yml exec -e CONFIRM_RESTORE="RESTORE GOALIX" worker npm run restore:db -- /app/backups/file.dump</pre>         | لما التطبيق مش قادر يفتح Settings.                                   |
| <b>تفعيل زر Restore في Settings</b> | <pre>BACKUP_RESTORE_ENABLED=true</pre>                                                                                                                     | مقفول افتراضيا في production. يحتاج password + عبارة RESTORE GOALIX. |
| <b>تغيير تكرار auto backup</b>      | <pre>BACKUP_INTERVAL_MINUTES=1440</pre>                                                                                                                    | القيمة الافتراضية يوميا. 15 يعني كل 15 دقيقة.                        |
| <b>تغيير مدة الاحتفاظ</b>           | <pre>BACKUP_RETENTION_DAYS=14</pre>                                                                                                                        | يحذف النسخ الأقدم عند تشغيل backup جديد.                             |
| <b>تعطيل auto backup</b>            | <pre>BACKUP_AUTOMATION_ENABLED=false</pre>                                                                                                                 | ال manual backup يظل متاحا.                                          |
| <b>فحص عدم commit dumps</b>         | <pre>npm run security:scan-dumps</pre>                                                                                                                     | يفشل لو dump files اتبعت في Git.                                     |
| <b>Sanitize dump للتسليم</b>        | <pre>cd golx-backend &amp;&amp; npm run dump:sanitize -- input.sql output.sql</pre>                                                                        | يمسح/يموه بيانات حساسة في plain SQL dumps.                           |

## 6. Data Lifecycle و Jobs

| الحالة / الهدف               | الأمر                                                                | ملاحظات التشغيل                     |
|------------------------------|----------------------------------------------------------------------|-------------------------------------|
| تقرير lifecycle              | <pre>cd golx-backend &amp;&amp; npm run data:lifecycle:report</pre>  | يعرض المرشحين للأرشفة.              |
| Dry run                      | <pre>cd golx-backend &amp;&amp; npm run data:lifecycle:dry-run</pre> | يختبر بدون تعديل فعلي.              |
| تشغيل lifecycle              | <pre>cd golx-backend &amp;&amp; npm run data:lifecycle:run</pre>     | ينقل/ينظف حسب السياسات.             |
| تفعيل lifecycle تلقائي       | <pre>DATA_LIFECYCLE_ENABLED=true</pre>                               | فعله على worker واحد فقط.           |
| تفعيل background automations | <pre>BACKGROUND_AUTOMATIONS_ENABLED=true</pre>                       | يفضل على worker وليس API.           |
| تفعيل injury risk automation | <pre>INJURY_RISK_AUTOMATION_ENABLED=true</pre>                       | يشغل scan أسبوعي/دوري حسب .env.     |
| تفعيل notification cleanup   | <pre>NOTIFICATION_CLEANUP_ENABLED=true</pre>                         | أرشفة وتنظيف notifications القديمة. |

## 7. الجودة والاختبارات والأمان

| الحالة / الهدف          | الأمر                                 | ملاحظات التشغيل                                                 |
|-------------------------|---------------------------------------|-----------------------------------------------------------------|
| Lint frontend           | <code>npm run lint</code>             | ESLint للواجهة.                                                 |
| Build frontend          | <code>npm run build</code>            | يتأكد أن Next production build شغال.                            |
| Type check              | <code>npx tsc --noEmit</code>         | مفيد قبل deploy.                                                |
| Backend lint            | <code>npm run backend:lint</code>     | يشغل lint داخل golx-backend.                                    |
| Backend tests           | <code>npm run backend:test</code>     | يشغل Jest backend.                                              |
| E2E tests               | <code>npm run test:e2e</code>         | Playwright smoke/e2e.                                           |
| Install browser for E2E | <code>npm run test:e2e:install</code> | يثبت Chromium لو ناقص.                                          |
| Security audit          | <code>npm run security:audit</code>   | npm audit للواجهة والباك.                                       |
| Quality checks          | <code>npm run quality</code>          | i18n + text + artifacts + dump scan                             |
| Full verify             | <code>npm run verify</code>           | lint + quality + build + backend<br>.lint/test + security audit |

## 8. Deploy وأدوات مساعدة

| الحالة / الهدف             | الأمر                                                        | ملاحظات التشغيل                     |
|----------------------------|--------------------------------------------------------------|-------------------------------------|
| <b>Auto deploy watch</b>   | <code>npm run deploy:auto</code>                             | يشغل scripts/auto-docker-deploy.ps1 |
| <b>Deploy مرة واحدة</b>    | <code>npm run deploy:auto:once</code>                        | مناسب لتحديث سريع.                  |
| <b>Linux deploy script</b> | <code>bash scripts/deploy-production.sh</code>               | لبيئة Linux/VPS إن كانت مهيأة.      |
| <b>تشغيل سكريبت CMD</b>    | <code>scripts\start-auto-docker-deploy.cmd</code>            | مفيد من ويندوز.                     |
| <b>Load test</b>           | <code>cd golx-backend &amp;&amp; npm run load:test</code>    | اختبار حمل أساسي.                   |
| <b>Mixed load test</b>     | <code>cd golx-backend &amp;&amp; npm run load:mixed</code>   | سيناريوهات مختلطة.                  |
| <b>Socket load test</b>    | <code>cd golx-backend &amp;&amp; npm run load:socket</code>  | يركز على realtime/socket.           |
| <b>Analyze load report</b> | <code>cd golx-backend &amp;&amp; npm run load:analyze</code> | تحليل نتائج الحمل.                  |

# سيناريوهات الطوارئ: لو حصل كذا اعمل كذا

| المشكلة                                   | التصرف الصحيح                                                                                           | أول أمر تبدأ به                                                               |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| الدانايز فاضية لكن الجداول موجودة         | شغل migrations لو محتاج، ثم bootstrap:admin، بعدها login وفقل MFA.                                      | <code>npm run bootstrap:admin</code>                                          |
| التطبيق مش بيفتح بعد deploy               | ابدأ بـ ps ثم logs ثم ready endpoint، وبعدها راجع .env secrets.                                         | <code>docker compose -f docker-compose.prod.yml ps</code>                     |
| Redis واقع                                | في dev شغل infra:up. في prod راجع logs redis وأعد تشغيل worker بعد رجوعه.                               | <code>npm run infra:up</code>                                                 |
| API شغال لكن jobs لا تعمل                 | راجع logs worker وBULLMQ_WORKERS_ENABLED وRedis.                                                        | <code>docker compose -f docker-compose.prod.yml logs -f worker</code>         |
| عايز نسخة قبل migration خطير              | اعمل backup يدوي ثم شغل migrate.                                                                        | <code>cd golx-backend &amp;&amp; npm run backup:db</code>                     |
| الدانايز اتمسحت والدخول مستحيل            | استرجع من CLI/worker لأن زر Settings لن يفتح بدون DB.                                                   | <code>CONFIRM_RESTORE="RESTORE GOALIX" npm run restore:db -- file.dump</code> |
| الأدمن نسي MFA أو الجهاز صاع              | استخدم إدارة MFA من admin آخر. لو مفيش admin آخر، استخدم bootstrap recovery بحذر أو تدخل DB مباشر مؤقت. | <code>BOOTSTRAP_ADMIN_ALLOW_EXTENSION=true npm run bootstrap:admin</code>     |
| ملفات dump ظهرت في Git                    | شغل scan-dumps واحذف الملفات من التتبع. لا ترفع production dumps.                                       | <code>npm run security:scan-dumps</code>                                      |
| عايز تعرض الموقع على موبايل في نفس الشبكة | شغل dev ثم lan:info وربما lan:setup.                                                                    | <code>npm run lan:info</code>                                                 |
| تغيير أسرار JWT أو CSRF                   | اتبع runbook rotation. لا تغير TOTP_ENCRYPTION_KEY مباشرة بدون re-encryption.                           | راجع docs/production-hardening-runbook.md                                     |

# أهم متغيرات البيئة

| المتغير                         | مثال / قيمة                                    | وظيفته                                      |
|---------------------------------|------------------------------------------------|---------------------------------------------|
| DATABASE_URL                    | postgresql://USER:PASSWORD@HOST:5432/golx_main | اتصال PostgreSQL.                           |
| REDIS_URL                       | redis://redis:6379                             | اتصال Redis للـ .cache/queues/realtime      |
| JWT_SECRET / JWT_REFRESH_SECRET | 32+ random chars                               | توقيع access/refresh tokens.                |
| CSRF_SECRET / COOKIE_SECRET     | 32+ random chars                               | حماية cookies و CSRF.                       |
| TOTP_ENCRYPTION_KEY             | 64 hex chars or 32-byte base64                 | تشفير أسرار MFA. لا تغيره مباشرة.           |
| MFA_ENFORCED_ROLES              | admin, coach                                   | يجبر هذه الأدوار على MFA.                   |
| BACKUP_DIR                      | /app/backups                                   | مكان حفظ ملفات dump و sha256.               |
| BACKUP_AUTOMATION_ENABLED       | true                                           | تشغيل backup تلقائي من worker.              |
| BACKUP_INTERVAL_MINUTES         | 1440                                           | الفاصل بين النسخ التلقائية.                 |
| BACKUP_RETENTION_DAYS           | 14                                             | حذف النسخ الأقدم.                           |
| BACKUP_RESTORE_ENABLED          | false                                          | تمكين زر restore في Settings في production. |
| BOOTSTRAP_ADMIN_PASSWORD        | StrongPass#123                                 | كلمة مرور أول أدمن أو recovery admin.       |
| BOOTSTRAP_ADMIN_ALLOW_EXISTING  | false                                          | اسمح بتحديث أدمن موجود عند recovery فقط.    |

## Production Deploy قبل Checklist

### نفذ بالترتيب:

1. تأكد من env secrets قوية وغير placeholder.
2. شغل migrations.
3. شغل bootstrap admin لو الداتا فاضية.
4. تأكد أن worker يعمل وأن Redis healthy.
5. أنشئ backup يدوي قبل أي تغيير كبير.
6. شغل verify أو على الأقل lint/build/backend:test.
7. اختبر login + MFA + dashboard + backup status.

.Generated from the current Goalix workspace scripts and operational runbooks